



第五章 函数

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院

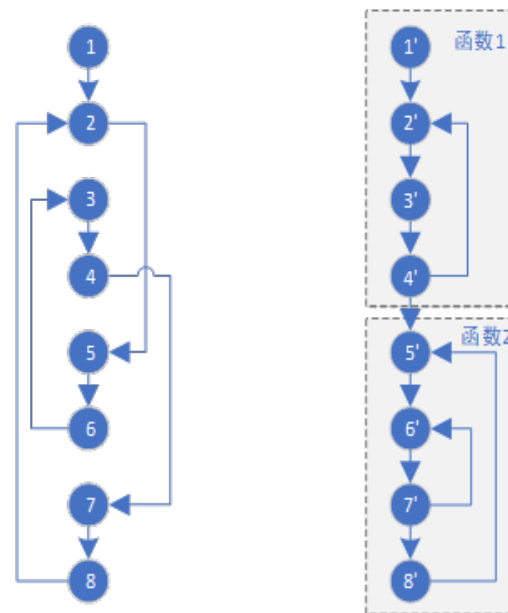


目录

1. 函数的引入
2. 函数定义
3. 函数调用
4. 变量作用域
5. 函数参数
6. 函数返回值
7. Lambda表达式
8. 文档字符串
9. 函数递归算法
10. 应用实践
11. 本章习题

1. 函数的引入

- **“goto” 语句的引入与弊端：**为处理代码内逻辑复杂性和频繁跳转，引入“goto”语句实现行间跳转，但此法使得跳转逻辑过于复杂，难以理解与维护
- **函数的引入改革：**为克服“goto”语句的缺陷，引入函数概念，将无序跳转转换为有序、可控的函数间调用，促进了编程结构化
- **结构化编程的实现：**通过定义重复使用的代码为函数，实现代码的结构化整理，将散落代码归入“函数房间”，提高了代码的组织性和可维护性。



(a) goto语句 (b) 函数调用



2. 函数定义

- **python内置的函数**：如print()、range()和input()等。这些内置函数的作用是方便用户编程，原本一个复杂任务可通过一个简单的函数调用即可完成
- **自定义函数**：除内置函数外，用户可自定义函数。使用函数一般需要先定义函数，然后调用函数。
- **定义函数语法**
 - **def** <函数名>(<参数列表>):
 <函数体>
 return <返回值列表>
 - 其中，def 是定义函数的关键字；<函数名>可以是任何有效的Python标识符；<参数列表>是调用该函数时传递给它的值，参数个数可以从零个到多个，多个参数时，参数之间用逗号隔开；<函数体>是实现函数功能的一行或多行代码；<返回值列表>是函数执行结束后，将零个、一个或多个返回值返回函数调用者，当没有返回值时，return语句可以省略。



函数定义

- 调用函数语法

<函数名>(<参数列表>)

- 调用函数中的<函数名>即为定义函数时的函数名，调用函数的<参数列表>是传给该函数的参数值，称为“**实际参数**”，简称“**实参**”（Arguments）。
- 与此相对，定义函数中的<参数列表>称为“**形式参数**”，简称“**形参**”（Parameters）。
- 在调用函数中的实参分别传递给定义函数中的对应的形参



代码示例：函数定义和使用

- # 定义函数

```
def add(num1, num2):  
    sum = num1 + num2  
    return sum
```

- # 调用函数

```
sum = add(10, 20)  
print(sum)
```



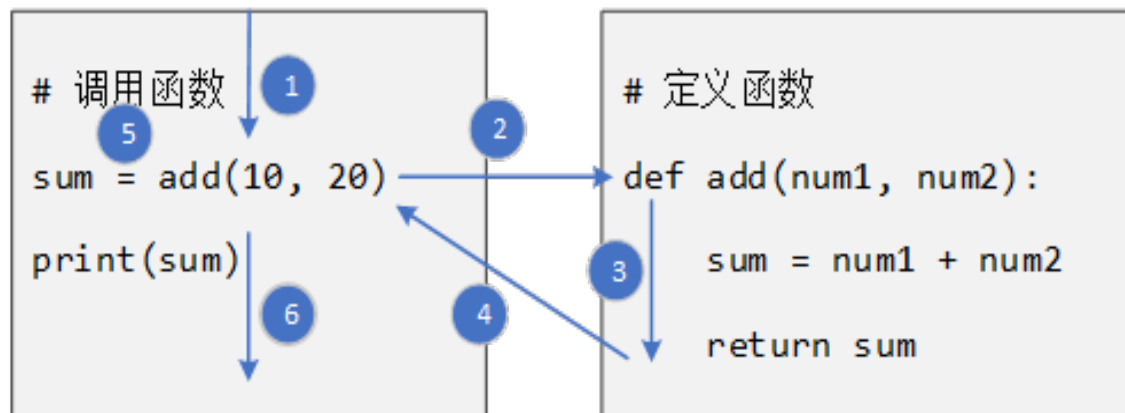
设计函数的原则

- 设计函数时，一般遵循如下原则：
 - 将可能需要重复执行的代码封装为函数，在需要时通过调用函数实现代码复用。
 - 确保函数功能单一且独立，避免在一个函数中实现过多功能，这样有助于提高函数的可重用性。
 - 保持函数内部代码的高内聚性，即函数内的逻辑紧密相关。
 - 保持函数之间的低耦合度，减少相互依赖性。



3. 函数调用

- 定义函数之后，通过调用函数来使用它提供的能力。
- 程序调用一个函数需要以下4个执行步骤：
 - 1. 调用程序在调用函数处暂停执行
 - 2. 将调用函数中的实参——复制给定义函数中的形参
 - 3. 程序转入函数体语句执行，直至函数执行结束，返回返回值
 - 4. 执行过程返回到调用程序处，获得函数返回值，继续调用程序的后续执行





5. 变量作用域

- 根据变量作用域，将变量分为**局部变量**和**全局变量**。
 - **局部变量**：指在函数内部声明的变量，其作用域（Scope）仅局限在这个函数内部，函数外部无法使用该变量，退出函数时变量将被自动删除。
 - **全局变量**：指在函数外部定义的变量，在函数内部或外部均可使用的变量，其作用域是整个程序，即**全局（Global）作用域**适用

```
count = 1
```

```
def read_global_count():  
    print('读取全局变量的值：', count)  
    # count = count + 1 # 如果没有注释掉这行语句，函数会报错么，为什么？
```

```
def increase_local_count():  
    count = 3 # 局部变量与全局变量同名，局部变量覆盖全局变量  
    count = count + 1  
    return count
```

```
read_global_count()  
print('全局变量的值：', count)  
print('函数返回局部变量的值：', increase_local_count())  
print('全局变量的值：', count)
```

```
读取全局变量的值： 1  
全局变量的值： 1  
函数返回局部变量的值： 4  
全局变量的值： 1
```



全局变量

- 在函数中使用全局变量有两种情况，**读全局变量**和**写全局变量**。
 - **读全局变量**：读一个全局变量意味着只读取，而不去修改它的值，这种情况一般出现在赋值符号“=”的右侧、或函数的实参等。在一个函数内部，可以直接读一个全局变量。
 - **写全局变量**：写一个全局变量意味着修改它的值，这时全局变量一般出现在赋值符号的左侧。在一个函数内部，写一个全局变量之前，首先需要用**global**关键字声明该变量为全局变量，然后才可利用赋值语句修改该全局变量的值。
- **使用global语句可以声明全局变量**
 - global关键字主要用于在函数内部修改全局变量的值，而对于只读访问全局变量则不需要使用global关键字
- **全局变量语法结构**
 - `var = 0` *# 声明一个全局变量*
 - `def fun():`
 - `global var` *# 声明该变量为全局变量*
 - ...



举例：全局变量

- 举例：读全局变量

- $PI = 3.14$

```
def circle_area(radius):
```

```
    area = PI * radius ** 2
```

```
    return area
```

```
print("The area of the circle:", circle_area(10))
```

- 举例：写全局变量

```
PI = 3.14
```

```
area = 0
```

```
def circle_area(radius):
```

```
    global area
```

```
    area = PI * radius ** 2
```

```
    print("The variable 'area' inside the function is ", area)
```

```
    return area
```

```
print("The area of the circle:", circle_area(10))
```

```
print("The variable 'area' outside the function is ", area)
```



5. 函数参数

- 函数内部的执行，往往需要函数外部的信息输入。这种信息输入一般有两种途径：**全局变量**和**函数参数**
 - **全局变量方式**：多个函数之间共享全局变量，实现了多个函数之间的信息共享，也实现了函数外部信息输入函数内部的目的，同样，也实现了函数内部信息输出到函数外部的目的。
 - 虽然全局变量方便且功能强大，但是，这导致了多个函数之间、或者多个函数内部与外部之间产生了复杂的关联关系，**增加了函数间的耦合度**，容易破坏程序的模块化设计原则。因此，**谨慎使用全局变量方式**
 - **函数参数**：函数参数提供了从**函数外部输入信息**至**函数内部**的标准途径。在定义函数阶段，设置函数的形参名称，在函数体中可以直接访问这些形参变量，而无需声明和定义。
 - 在调用函数阶段，从函数外部将实参变量传输给形参变量，实现信息的输入。**函数参数方式保证了函数之间调用关系、或依赖关系清晰，有利于保证程序的模块化设计原则**



参数默认值

- 在定义函数时，可以为参数（形参）设置默认值，这样在调用函数时可以省略此参数的赋值，Python 会自动使用设定的默认值。
- 默认值建议设为不可变对象，如整数、字符串等。当提供实参时，该实参会覆盖默认值。通过设置默认值，可以在一定程度上简化函数调用。

- **举例**

- `def person(name, nationality='中国'):`
 `print('姓名: {0}, 国籍: {1}'.format(name, nationality))`
 `person('小明')`
 `person('Tom', 'USA')`



关键字参数

- 当函数有多个参数时，参数传递有两种主要方式。
 - **位置参数 (Positional Arguments)**：按照定义时的顺序传递参数，即调用函数时参数的位置要与定义顺序一致。
 - **关键字参数 (Keyword Arguments)**：在调用函数时，显式指定参数的名称，允许在任何顺序中传递参数。这种方式可以提高代码的可读性和明确性。
- **举例1：**
 - ```
def location(x, y=20, z=30):
 print('x=', x, ';y=', y, ';z =', z)
location(50, 60, 70)
location(z=100, x=80)
```
- 注意，这里的“参数”指的都是实参 (Arguments)。对于形参 (Parameters) 来说，在定义时必须同时指定每个形参的位置以及名称。因此，一般不会有“ Positional Parameters” 或者“ Keyword Parameters” 这样的称谓。



# 混合使用位置参数和关键字参数

- 当在调用函数时，位置参数和关键字参数可以混合使用
  - 但是需要注意，所有的位置参数必须出现在关键字参数之前
  - `def greet(name, greeting="Hello"):`  
    `print(f"{greeting}, {name}!")`  
    `greet("Alice", greeting="Hi")`



# 默认参数值的陷阱

- 默认值为可变对象（如列表或字典）时，可能会引发意想不到的行为。默认参数值在函数定义时创建，会在每次调用中复用该对象。
- **举例**
  - ```
def append_to_list(value, my_list=[]):  
    my_list.append(value)  
    return my_list  
print(append_to_list(1)) # 期望输出 [1]  
print(append_to_list(2)) # 期望输出 [2], 但实际输出 [1, 2]
```
 - 理解这个陷阱的关键在于：明白 Python 函数的默认参数值是在函数定义时计算并创建的，而不是在函数调用时



默认参数值的陷阱

- 为了避免默认参数值的陷阱，**建议不要使用可变对象作为函数参数的默认值，而是使用 `None` 作为默认值，并在函数内部检查这个值：**

- **def** `append_to_list(value, my_list=None)`:
 - # 如果没有提供 `my_list`, 创建一个新的空列表*
 - if** `my_list is None`:
 - `my_list = []`
 - `my_list.append(value)`
 - return** `my_list`

```
result1 = append_to_list(1)
print(result1) # 期望输出 [1]
result2 = append_to_list(2)
print(result2) # 期望输出 [2]
```



可变数量参数

- Python允许定义的函数有**任意数量的参数**，方便用户调用函数。
- 定义函数时，在参数前增加星号（*）或两个星号（**）实现
 - **星号参数**：带有星号的可变参数，从此处开始直至结束的所有的**位置参数**都将被汇集成为一个**元组**（tuple），星号参数只能出现在没有星号参数列表的后面
 - **双星号参数**：带有双信号的**可变参数**，从此处开始直至结束的所有**关键字参数**都将被汇总称为一个**字典**（dictionary）
- **语法结构**
 - **def** 函数名(param1, *param2, **param3):
 <函数体>
- **扩展阅读**: <https://stackabuse.com/variable-length-arguments-in-python-with-args-and-kwargs/>
 - **def** super_function(num1, num2, *args, callback=None, messages=[], **kwargs):
 - 用解包（Unpacking）语法来传递参数



举例：可变数量参数

- **举例：**利用可变数量参数实现酒店预订

```
def reserve_hotel(name, *room_numbers, **amenities):
```

```
    # 功能是预订酒店房间, room_numbers 是房间编号集合, amenities 是房间的设施
```

```
    print('Hotel name:', name)
```

```
    for room_number in room_numbers:
```

```
        print('Room number:', room_number)
```

```
    for amenity, quantity in amenities.items():
```

```
        print(f"{amenity}: {quantity}")
```

```
    reserve_hotel('Hotel A', 101, 102, beds=2, TV=1)
```

- **与AI协作探索：**Python中，什么场景下需要定义可变数量个参数？这个功能可以帮助我们实现哪些复杂设计？



序列解包

- 如何向接受可变数量参数的函数便捷地传递大量参数？
- **序列解包 (unpacking)**：一种将一个可迭代对象（如列表、元组、字典、字符串等）解开，将其中的值分别赋给多个变量的机制。其应用场景包括：
 - **(1) 变量赋值**：在变量初始化时，可直接从列表或元组等数据结构中提取元素值
 - `x, y, z = 1, 2, 3`
 - `tuple1 = ('shanghai', 2024, 6341)`
 - `city, year, area = tuple1`
 - **(2) 函数返回值处理**：函数可以返回多个值（实际上以元组形式），通过序列解包，可以将这些值分别赋给不同的变量
 - `def get_coordinates():`
 - `return (10, 20)`
 - `x, y = get_coordinates() # x=10, y=20`



- **(3) 交换变量值**: 利用解包快速交换变量值, 无需临时变量
 - `a, b = 5, 10`
 - `a, b = b, a # a=10, b=5`
- **(4) 忽略不需要的变量**: 用下画线 `_` 忽略无关元素
 - `data = (1, 2, 3, 4, 5)`
 - `first, _, third, _, fifth = data # first=1, third=3, fifth=5`
- **(5) 带 * 的拆包**: 星号表达式用于接收剩余的多个元素, 通常用于不定长序列。
 - `car_prices = [50, 26, 14, 80, 150, 10, 5, 30]`
 - `highest, *others, lowest = sorted(car_prices, reverse=True)`
 - `# highest=150, lowest=5, others=[80, 50, 30, 26, 14, 10]`



嵌套解包

- **嵌套解包**：支持对复杂结构（如嵌套列表、嵌套元组）的元素直接进行分配
 - `nested_list = [(1, 'apple'), (2, 'orange')]`
 - `for number, fruit in nested_list:`
 - `print(f"Number {number} is for {fruit}")`
- **解包在函数调用中的应用**：在函数调用中，使用 `*` 和 `**` 操作符可以实现序列和字典的解包，分别传递位置参数和关键字参数
 - **(1) 使用 `*` 解包位置参数，将序列解包并传入函数**
 - `def func(x, y, z):`
 - `print(x, y, z)`
 - `tuple_args = (1, 2, 3)`
 - `func(*tuple_args)` # 输出: 1 2 3
 - **(2) 使用 `**` 解包关键字参数，将字典解包传入函数**
 - `dict_args = {'x': 1, 'y': 2, 'z': 3}`
 - `func(**dict_args)` # 输出: 1 2 3



多序列操作

- **zip()函数**：将多个可迭代对象中的对应元素打包成一个个元组，以便并行遍历

- `keys = ['广东','山东','河南']` #省份列表
- `values = [126012510, 101527453, 99365519]` #2020年人口数
- `for k, v in zip(keys, values):`
- `print(k,v)`



- `zip()` 返回一个迭代器，用完即失效，若需重复使用，需转化为列表或其他结构
- `zip` 取名自英文单词 `zip`，有“把.....的拉链拉上或拉开”的含义：
 - 拉链有左右两排齿，拉头经过时，左右齿一一咬合、配对
 - `zip()` 把多个可迭代对象（列表 / 元组等）按位置逐项配对，生成一个个**元组**
- `zip()`的逆操作：
 - `coupled_list = list(zip(keys, values))` # `[('广东', 126012510), ...]`
 - `keys, values = list(zip(*coupled_list))` # `('广东','山东','河南' ), (126012510, 101527453, 99365519)`



6. 函数返回

- **函数参数**：实现从函数外部到函数内部的信息传递；**函数返回**：实现从函数内部到函数外部的信息传递。
- 在函数定义代码中，采用return语句退出函数并将程序返回到调用函数的位置继续执行。**return语句可以同时返回0个、1个或多个函数结果**
- (1) **无返回值**：当函数无需返回任何值时，可省略return语句。这时在函数末尾隐含了一句return None
 - `def print_hello(name):`
 `print('Welcome ', name)`
 `print_hello('小明')`
- (2) **1个返回值**：函数返回一个值
 - `def add(num1, num2):`
 `return num1 + num2`
 `print(add(10,20))`
- **思考**：如何定义可变数量参数，实现任意数量数字的加和？



函数返回

- (3) **多个返回值**: 当函数返回多个值时, 这多个值以元组类型组织并返回
 - `def swith(first, second) :`
 `return second, first`
 `print(swith('a','b' ))`
- 在实际应用中, 如果返回值数量超过四个, 代码的可读性和维护性可能下降。这时可以使用命名元组、类或字典来明确表述每个返回值的含义, 便于代码的后续维护



思考-博物馆名画失窃案

- 博物馆的一幅名画被盗，警方锁定了 5 名嫌疑人：A, B, C, D, E。经过调查，警方确认：
 - 这 5 人中共有 2 名小偷。
 - 每名嫌疑人都提供了 2 条证词，每条证词既可能是真话，也可能是假话。
 - 警方通过技术手段确认，在这 10 条证词中，恰好有 6 条是真话，4 条是假话。
 - 已知所有小偷之间达成了攻守同盟，他们对任何一名嫌疑人（包括自己）的身份定性必须保持一致。
- 审问记录如下：
 - A 说：B 是小偷，C 也是小偷。
 - B 说：A 是清白的，E 是小偷。
 - C 说：D 是小偷，A 也是小偷。
 - D 说：C 是小偷，E 是清白的。
 - E 说：自己是清白的，C 也是清白的。
- 编写一个 Python 程序，通过逻辑枚举找出哪两个人是小偷。



6. Lambda表达式

- 当函数的功能简单到只需要一个表达式实现时，Python提供了一种匿名函数方式，即**Lambda表达式**
 - **Lambda表达式语法**
`<函数名> = lambda <参数列表> : <表达式>`
 - 其中参数列表支持参数默认值和关键词参数。lambda表达式等价于下面的标准函数形式：

```
def <函数名>(<参数列表>):  
    return <表达式>
```
 - lambda表达式返回一个函数类型

```
f = lambda x,y,z: x+y+z  
print(type(f))  
print(f(1,2,3))
```
 - 输出结果

```
<class 'function'>  
6
```
 - 输出结果显示变量f是一个function类型
- lambda表达式并非仅仅**减少编程代码量**，更重要的在Python的一些内置函数或自定义函数中**作为这些函数的参数**，代码灵活而优雅
- **与AI协作探索**：函数本身可以被视作数据么？可以作为参数传递么？可以作为返回值返回么？



7. 文档字符串

- 编程语言引入了**文档字符串** (Documentation Strings, 简称 **docstrings**) 的概念。通过特定格式书写的解释性文字，文档字符串可以被自动提取生成程序文档，方便开发者参考
- 在函数中的文档字符串，**一般放置于函数中的第一行，用三引号 (""") 括起来**，该字符串一般说明该函数的功能、参数、异常处理等说明性文字。**注意文档字符串与注释类似，仅供人类查看，程序不执行**

- **语法格式**

- **def fun(param1):**

- """解释函数功能、函数参数等文字"""

- # 其他程序代码

- 自动化工具可以通过函数的属性 **__doc__** 来获取文档字符串 (注意doc前后均为双下划线)

- **举例**

- **def max(a, b):**

- """从两个输入的数字中，找出最大的数字"""

- if a >= b:**

- return a**

- else:**

- return b**

- print(max.__doc__)**



8. 字符串处理

- **字符串处理函数**: 字符串是编程中用于存储和表示文本的基本数据类型。在 Python 中, str 类型提供了一系列的内置方法, 允许开发者执行各种字符串操作, 从基本的文本处理到复杂的模式匹配
- 常用的字符串处理函数及其用法:
- **format()**: 用于将指定的值插入字符串的占位符中。占位符由花括号 {} 表示。
 - `text = "The number is: {}".format(42)`
 - 运行结果: The number is: 42
- **join()**: 连接序列中的元素, 如列表, 以传入的字符串作为分隔符。
 - `words = ['Hello', 'World']`
`sentence = ' '.join(words)`
 - 运行结果: Hello World
- **split()**: 将字符串拆分为列表, 使用指定的分隔符。如果没有指定分隔符, 默认使用空格分隔。
 - `sentence = 'Hello World'`
`words = sentence.split(' ')`
 - 运行结果: ['Hello', 'World']



实践：字符串处理

- **replace()**: 在字符串中查找指定的子串并替换为另一个字符串。

```
text = 'bat bat symphony'
replaced_text = text.replace('bat', 'cat')
```

运行结果: cat cat symphony

- **strip()**: 删除字符串开头和结尾的空格或指定的字符。

```
text = ' Hello World '
```

```
stripped_text = text.strip()
```

运行结果: Hello World

- **upper()**: 将字符串中的所有字符转换为大写。

```
text = 'hello world'
upper_text = text.upper()
```

运行结果: HELLO WORLD

- **lower()**: 将字符串中的所有字符转换为小写。

```
text = 'HELLO WORLD'
lower_text = text.lower()
```

运行结果: hello world

- **capitalize()**: 将字符串的首个字符转换为大写, 其余字符转换为小写。

```
text = 'hello world'
capitalized_text = text.capitalize()
```

运行结果: Hello world

- **title()**: 将字符串中每个单词的首字母转换为大写, 其余字母转换为小写。

```
text = 'hello world'
title_text = text.title()
```

运行结果: Hello World

- **find()**: 搜索子串第一次出现的索引位置, 如果没有找到子串, 返回 -1。

```
text = 'hello world'
index = text.find('world')
```

运行结果: 6

- **endswith()**: 检查字符串是否以指定后缀结束, 返回布尔值。

```
filename = 'report.pdf'
is_pdf = filename.endswith('.pdf')
```

运行结果: True

- **startswith()**: 检查字符串是否以指定前缀开始, 返回布尔值。

```
url = 'http://example.com'
starts_with_http = url.startswith('http://')
```

运行结果: True



9. 函数递归算法

- **函数递归算法：**在函数定义中，函数不仅可以调用其他函数，还可以调用自身。这种在函数定义中调用自身的方式称为函数递归。
- 递归广泛用于解决数学和计算领域中的一类特殊问题：**将问题分解为规模更小的同类子问题来解决。**
- **递归的思想与数学归纳法相似：**证明当 n 等于任意一个自然数时某命题成立。证明分下面两步：
 - 证明当 n 取第一个数 n_0 时命题成立
 - 假设 n 取 n_k ($k \geq 0$, k 为自然数)时命题成立，证明当 $n = n_{(k+1)}$ 时命题也成立
- **解决递归问题，掌握三个关键要素：**
 - 利用迭代方法，将一个大问题转化为一个或几个子问题
 - 明确递归结束的终止条件；如果没有终止条件，将会导致程序无限递归的情况
 - 给出终止条件下不再迭代，而是直接计算出结果



举例：阶乘求解

- 编程实现阶乘： $n! = n(n-1)(n-2)\dots 1$
- 将阶乘转化为递归的形式
- (1) 当 $n=0$, $n!=1$
- (2) 当 $n>0$, $n!=n*(n-1)!$

```
def factorial(n):
```

```
    print("calling factorial({0})".format(n) )
```

```
    if n == 0:
```

```
        print("factorial({0}) returns {1}".format(n, 1))
```

```
        return 1
```

```
    else:
```

```
        value = n * factorial(n-1)
```

```
        print("factorial({0}) returns {1}".format(n, value))
```

```
        return value
```

```
fact = factorial(3)
```

```
print("Result: factorial({0}) = {1}".format(3,fact))
```




递归与循环

- **递归与循环的比较共同点：**

- 都是通过反复执行代码来完成任务
- 用于解决具有重复性特点的任务

- **基本概念：**

- **递归：**通过反复调用同一个函数来实现循环
- **循环：**通过反复执行循环体中的代码段来实现循环

- **算法设计**

- 递归和循环在算法设计上并无绝对的优劣之分

- **实际应用建议**

- 当任务既可通过递归也可通过循环解决时：如果程序设计复杂度相当，优先考虑使用循环



闭包 (Closure) —— 让函数拥有“记忆”

- 闭包是指一个**内部函数**引用了其**外部嵌套函数**作用域中的变量。即使外部函数已经执行完毕并退出，内部函数依然能够“记住”并访问这些变量。
- 闭包的构成条件：
 - 必须有嵌套函数：在一个函数内部定义另一个函数。
 - 必须引用外部变量：内部函数使用了外部函数的局部变量。
 - 必须返回内部函数：外部函数将这个内部函数作为返回值返回。

```
def make_linear_fun(w, b): # 外部函数 (工厂)
    def linear(x):         # 内部函数 (闭包)
        # 内部函数“记住”了外部传入的 w 和 b
        return x * w + b
    return linear          # 返回这个带有记忆的函数
```

生产一个：斜率为 2, 截距为 5 的线性函数

```
linear_fun = make_linear_fun(2, 5)
```

调用这个闭包

```
print(linear_fun(10)) # 输出 25 (计算过程:  $10 * 2 + 5$ )
```

- 为什么使用闭包？
 - 数据封装：外部无法直接修改 w 和 b ，这起到了一种类似“私有变量”的保护作用。
 - 减少全局变量：避免为了保存状态而定义过多的全局变量，从而防止全局污染。
- 普通函数 vs. 闭包
 - 普通函数：执行时创建局部变量，执行完后变量随之销毁。
 - 闭包：将外部变量“捕获”到了自己的环境包（Environment）中，这些变量会随函数对象一起持久存在。



思考：实现神经网络线性层

- 在神经网络中，一个线性层（Linear Layer）执行的操作是：

$$y = Wx + b$$

- 其中 $x \in R^{d_{in}}$ 是输入向量， $y \in R^{d_{out}}$ 是输出向量， $W \in R^{d_{out} \times d_{in}}$ 是权重矩阵， $b \in R^{d_{out}}$ 是偏置项。
- 请定义一个**工厂函数** `make_linear_layer(weights, bias)`。该函数应具备以下特性：
 - 参数接收**：接收形状为 $d_{out} \times d_{in}$ 的权重矩阵 `weights` 以及长度为 d_{out} 的偏置项 `bias`。`weights` 以嵌套列表的形式提供：外层列表元素个数为 d_{out} ，每个元素对应一个内层列表，每个内层列表包含元素个数为 d_{in} 。`bias` 以列表的形式提供。
 - 闭包构建**：返回一个内部函数 `linear(x)`。这个内部函数能够“记住”外部传入的权重和偏置。
 - 矩阵运算**：`linear(x)` 接收一个以列表形式提供的输入向量 x ，通过列表推导的方式完成计算，并以列表形式返回线性变换后的结果 y 。



案例1：等额本息还款

- **等额本息还款**，是指贷款人每月以相等的金额按期归还贷款本金和利息，直至贷款期满。这种方式的优点是每月的还款金额固定，便于贷款人做出还款计划
 - 等额本息还款的核心原理在于每月的还款额度中，既包括了一部分本金，也包括了一部分利息，且整体金额保持一致。
 - $\text{月还款额} = [\text{贷款本金} \times \text{月利率} \times (1 + \text{月利率})^{\text{还款月数}}] \div [(1 + \text{月利率})^{\text{还款月数}} - 1]$

```
def calculate_monthly_payment(principal, annual_interest_rate, loan_years):
```

```
    number_of_payments = loan_years * 12
```

```
    monthly_interest_rate = annual_interest_rate / 12
```

```
    monthly_payment = (principal * monthly_interest_rate * (1 + monthly_interest_rate) ** number_of_payments) /  
                        ((1 + monthly_interest_rate) ** number_of_payments - 1)
```

```
    return monthly_payment
```

```
# 示例：如果你贷款100000元，年利率是5%，贷款期限是10年
```

```
principal = 100000
```

```
annual_interest_rate = 0.05
```

```
loan_years = 10
```

```
monthly_payment = calculate_monthly_payment(principal, annual_interest_rate, loan_years)
```

```
print(f"如果你贷款 {principal} 元，年利率 {annual_interest_rate*100}%，贷款期限 {loan_years} 年，每月需要还款：  
{monthly_payment:.2f} 元。")
```



案例2：股票期望回报率与风险

- 根据股票的贝塔系数 (Beta) 计算其期望回报率:

- 期望回报率=无风险回报率+Beta*(市场回报率-无风险回报率)

```
def expected_return(risk_free_rate, beta, market_return):
```

```
    return risk_free_rate + beta * (market_return - risk_free_rate)
```

```
expected_r = expected_return(0.03, 1.2, 0.08)
```

```
print(f"期望回报率为: {expected_r*100:.2f}%")
```

- 用lambda表达式实现

```
expected_return = lambda risk_free_rate, beta, market_return: risk_free_rate + beta *  
(market_return - risk_free_rate)
```

```
expected_r = expected_return(0.03, 1.2, 0.08)
```

```
print(f"期望回报率为: {expected_r*100:.2f}%")
```



案例3 学生课程管理系统

- 设计并实现一个学生课程管理系统，以管理学生的选课情况。具体功能包括：1) 允许学生注册和退选课程。2) 查询学生的已选课程。3) 统计每门课程的注册学生人数。
 - (1) 模拟学生和课程信息
 - `students = ["xiaozhang", "xiaowang", "xiaoli", "xiaozhao"]`
 - `courses = ["Math", "Physics", "Chemistry", "Biology"]`
 - (2) 初始化学生课程注册信息
 - `registration = {`
 - `"xiaozhang": {"Math", "Physics"},`
 - `"xiaowang": {"Math", "Chemistry"},`
 - `"xiaoli": {"Physics", "Biology"},`
 - `"xiaozhao": {"Math", "Biology", "Chemistry"}`
 - `}`



- (3) 学生注册课程
- `def register_course(student, course):`
- `if student in registration:`
- `registration[student].add(course)`
- `else:`
- `registration[student] = {course}`
- `print(f"{student} has successfully registered for {course}.")`
- (4) 学生退选课程
- `def drop_course(student, course):`
- `if student in registration and course in registration[student]:`
- `registration[student].remove(course)`
- `print(f"{student} has successfully dropped {course}.")`
- `else:`
- `print(f"{student} is not registered for {course} or {course} does not exist.")`



- (5) 查询学生的已选课程
 - `def get_registered_courses(student):`
 - `if student in registration:`
 - `return registration[student]`
 - `else:`
 - `return f"{student} has not registered for any courses."`
- (6) 统计每门课程的注册人数
 - `def course_registration_count():`
 - `course_count = {}`
 - `for courses in registration.values():`
 - `for course in courses:`
 - `if course in course_count:`
 - `course_count[course] += 1`
 - `else:`
 - `course_count[course] = 1`
 - `return course_count`



- 调用函数的示例操作：
 - `register_course("xiaozhang", "Biology")`
 - `drop_course("xiaozhao", "Math")`
 - `print(f"xiaozhang's courses: {get_registered_courses('xiaozhang')}")`
 - `print(f"Course registration count: {course_registration_count()}")`
 - `print(f"Common courses between xiaozhang and xiaowang: {common_courses('xiaozhang', 'xiaowang')}")`



本章习题

一、理论知识

1. 请解释变量作用域的概念。局部变量和全局变量的区别是什么？
2. 什么是关键字参数和可变参数？在调用函数时如何使用这两种参数？
3. 什么是面向过程的编程？面向过程的程序的执行实际上就是函数之间调用序列，每次运行程序，其中的函数是否都获得运行，为什么？
4. 请简述 Lambda 表达式的用途及其与普通函数的区别。
5. 比较循环和递归的异同点。

二、编程实践

- 1. 编写一个加减乘除四个函数，并在一个主函数main()中调用它们。
- 2. 编写一个函数 sum_all(*args)，可以接收任意数量的数字并返回它们的和。
- 3. 回文是指一个字符串从左往右读和从右往左读完全相同。编写一个函数 is_palindrome(s)，判断输入的字符串是否为回文。例如，输入"madam"，返回 True。
- 4. 编写两个函数 gcd(a, b) 和 lcm(a, b)，分别计算两个整数的最大公约数和最小公倍数。提示: 可以使用辗转相除法（欧几里得算法）求解最大公约数，然后利用公式 $lcm(a, b) = abs(a * b) // gcd(a, b)$ 计算最小公倍数。
- 5. 编写一个函数 caesar_cipher(s, shift)实现凯撒加密算法，对字符串 s 中的每个字母进行位移加密（凯撒加密）。shift 表示字母的位移量。仅加密字母字符，非字母字符保持不变。例如caesar_cipher("abc", 3) 应返回 "def"。提示：使用 ord() 函数获取字符的ASCII码值，计算新的字符位置，然后用 chr() 函数将新的ASCII码值转换为字符。
- 6. 创建一个Lambda表达式来计算根据输入的利润发放的奖金。假设奖金计算规则如下：如果利润小于或等于100,000元，则奖金比例为10%；如果利润超过100,000元但小于或等于200,000元，则超过100,000元的部分按20%计算奖金；如果利润超过200,000元，则超过200,000元的部分按30%计算奖金。
- 7. 编写程序，根据给定的年收入，使用累进税率计算个人所得税。所得税按月应纳税所得额分级，适用不同的税率和速算扣除数。具体税率表如下。

级数	全年应纳税所得额	税率（%）	速算扣除数
1	不超过36000元的	3	0
2	超过36000元至144000元的部分	10	2520
3	超过144000元至300000元的部分	20	16920
4	超过300000元至420000元的部分	25	31920
5	超过420000元至660000元的部分	30	52920
6	超过660000元至960000元的部分	35	85920
7	超过960000元的部分	45	181920



The End !